Graph traversals

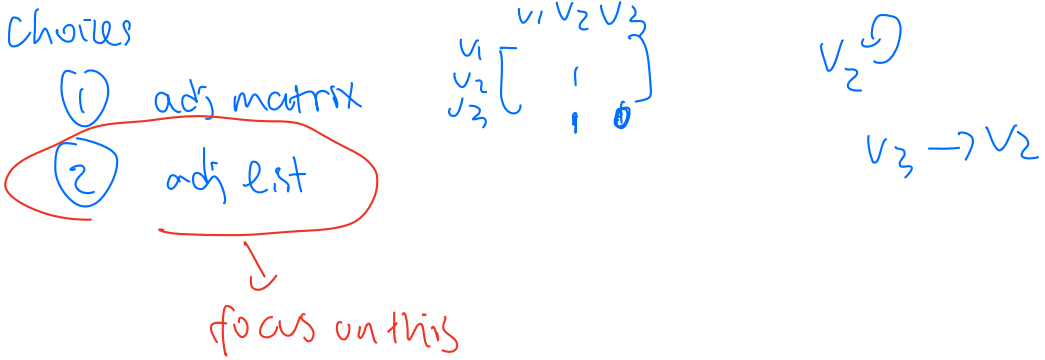
Vertices and edges

Undirected



Directed





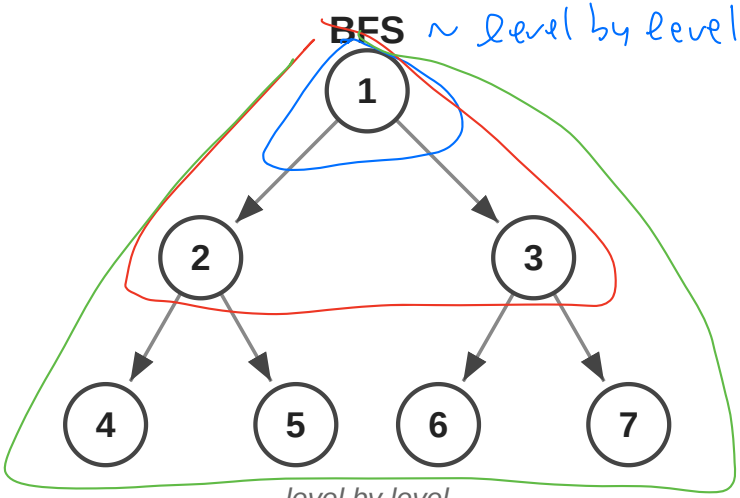
Graph representation

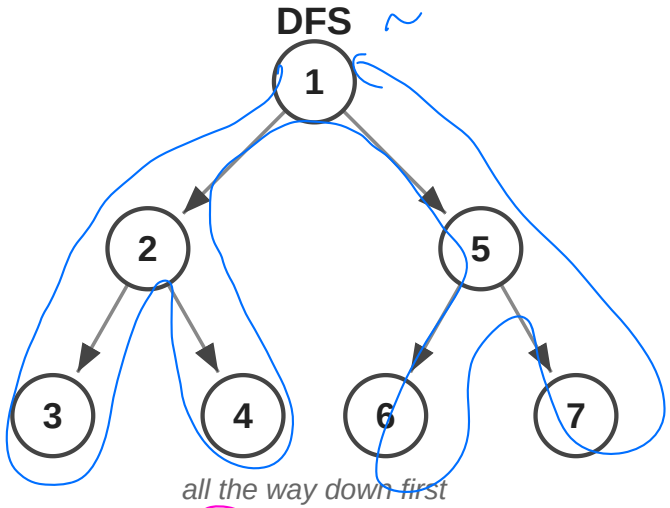
```
# adjacency list -- a dict from node to its out-neighbors
adj = {
  0: [1, 5],
  1: [2, 5],
  2: [3, 4],
}
```

from → to1, to2, ...



Traversing a graph: BFS, DFS





Q: shortest path in unweighted graph? A: BFS / DFS

↳ each node

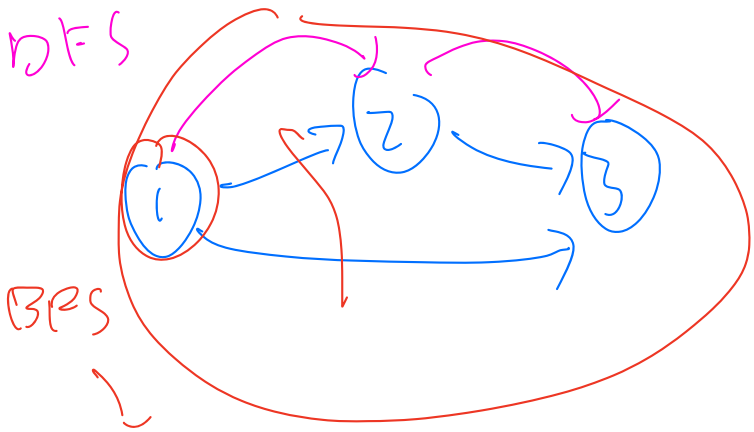
Q: then why dijkstra? A: weighted graph

Q: Runtime A: $O(V+E)$

BFS + DF

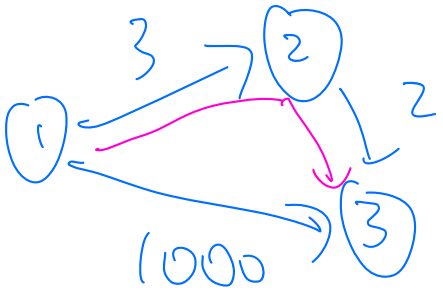
V = # nodes / vertices

E = # edges

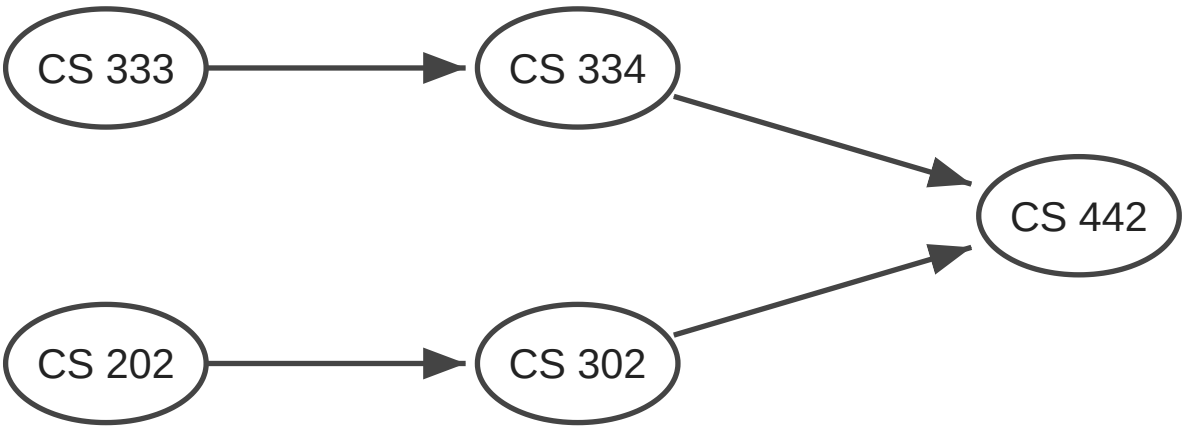


expand / distance at a time

Dijkstra.

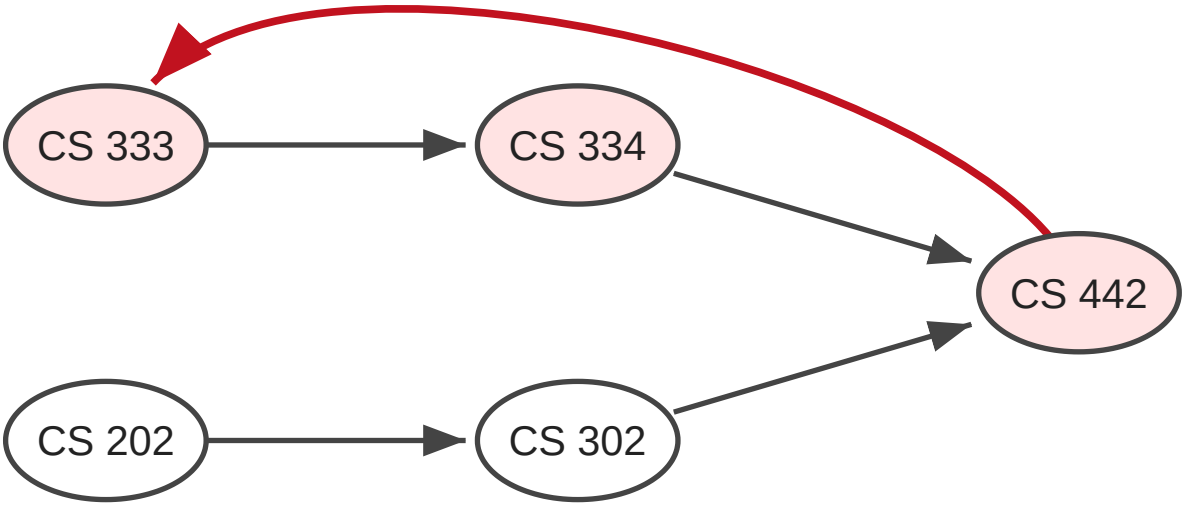


Problem: Course Prereq



an arrow means “must be taken before”

Cycle detection



an arrow means “must be taken before”

Cycle detection

Input: a directed graph.

Output: does it contain a **cycle**?

Approaches - topological sort

to DFS

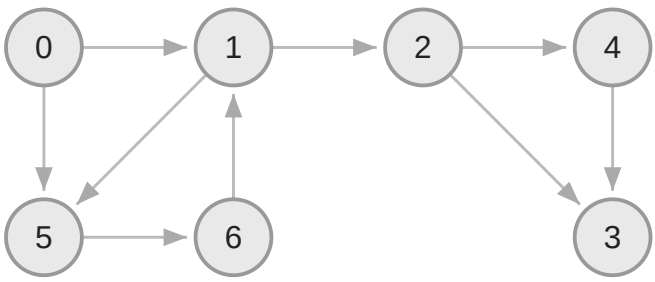
Depth-first Search

Node state

- **not visited** -- haven't touched it
- **visiting** -- it's on the stack *right now*, we're still inside it
- **done** -- finished it and everything below it

Current path: Stack<Node>. A straight path from starting node to current node.

DFS, one step at a time

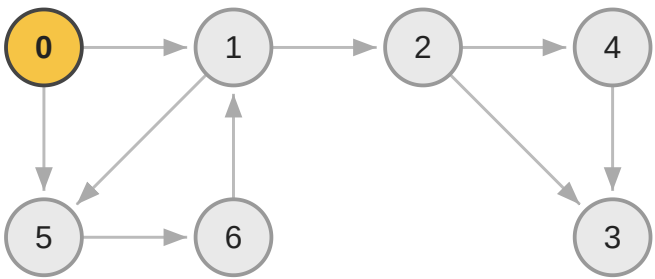




stack

(empty)

DFS, one step at a time



not visited

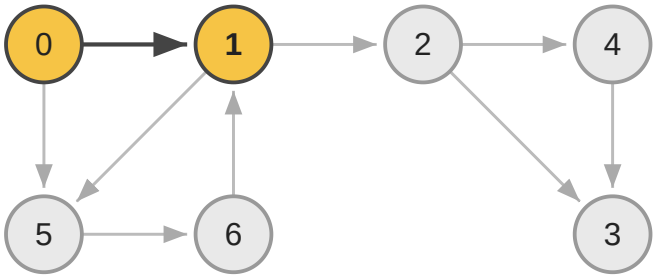
visiting (on stack)

done

stack

0

DFS, one step at a time

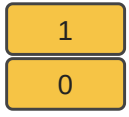


not visited

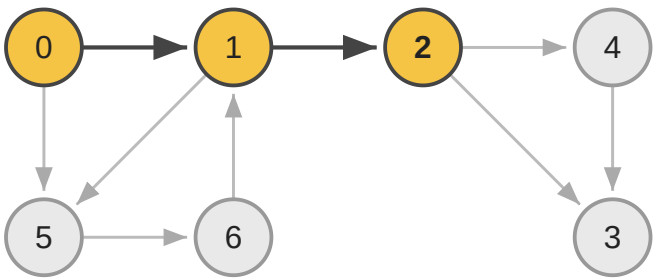
visiting (on stack)

done

stack



DFS, one step at a time

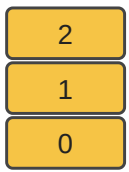


not visited

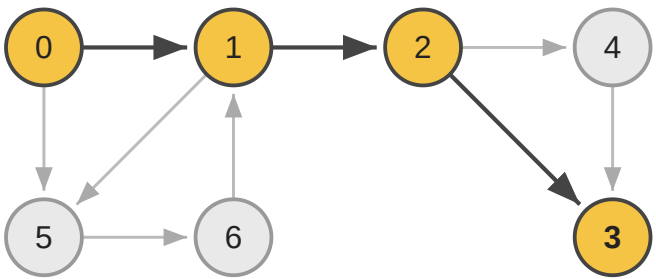
visiting (on stack)

done

stack



DFS, one step at a time



not visited

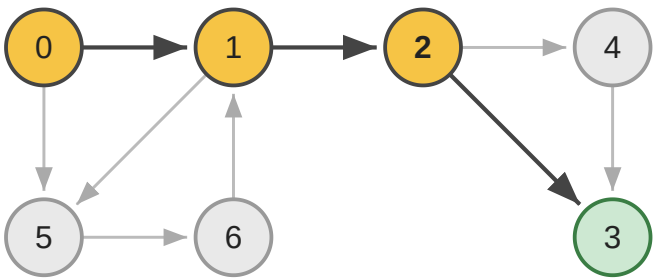
visiting (on stack)

done

stack



DFS, one step at a time

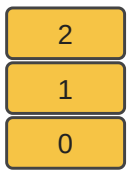


not visited

visiting (on stack)

done

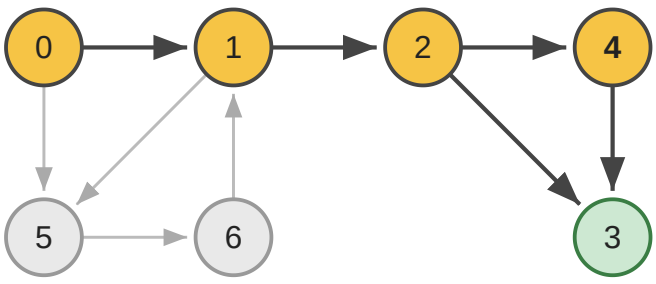
stack



Q: cycle condition? A: when you see a yellow node / node on my path

NVJ green.

DFS, one step at a time



not visited

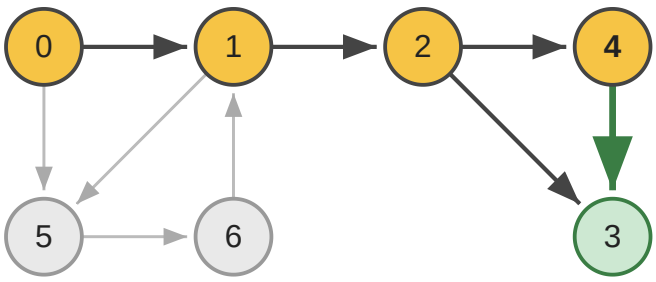
visiting (on stack)

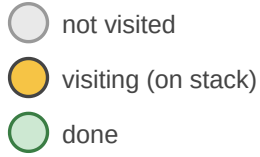
done

stack

4
2
1
0

"Already visited" is not the same as "cycle"

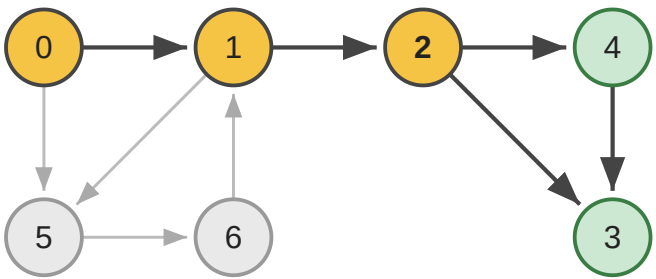




stack



DFS, one step at a time

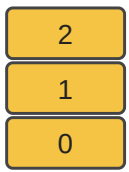


not visited

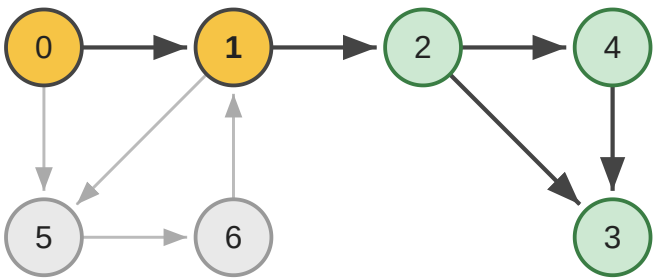
visiting (on stack)

done

stack



DFS, one step at a time



not visited

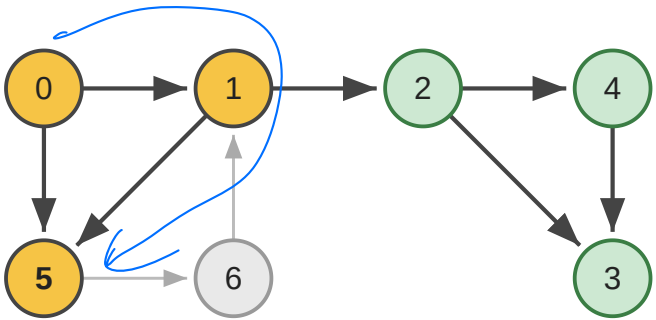
visiting (on stack)

done

stack



DFS, one step at a time



not visited

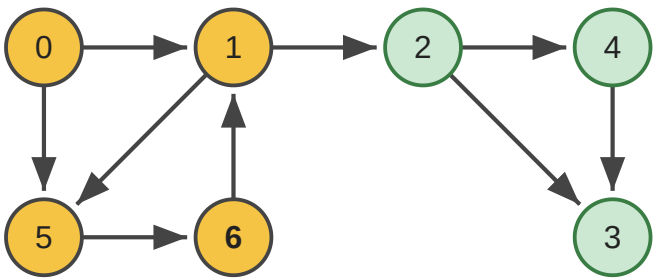
visiting (on stack)

done

stack



DFS, one step at a time



not visited

visiting (on stack)

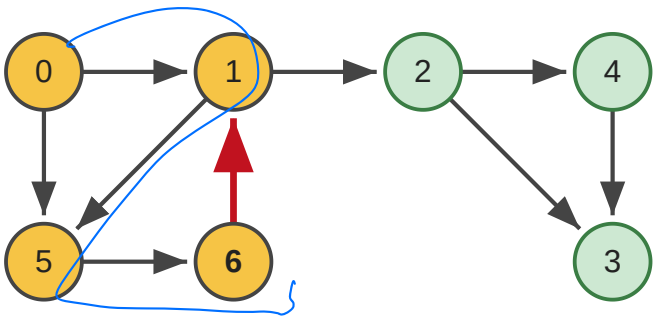
done

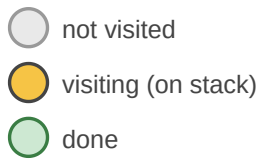
stack

6
5
1
0

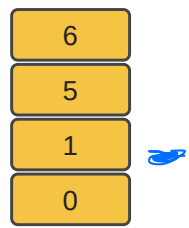
yellow -> yellow.

A cycle = an edge into your own stack





stack



Q: how to print cycle nodes? A: 1 -> 5 -> 6

↳ where the cycle was detected

DFS code template

```
# Gray, yellow, green
UNVISITED, VISITING, VISITED = 0, 1, 2
adj = {}
state = []
cycle_found = False

def dfs(cur_node):
  if cycle_found:
    return
  # VISITING = on the stack right now
  state[cur_node] = VISITING

  for next_node in adj[cur_node]:
    # an edge into the stack is a cycle
    if state[next_node] == VISITING:
      cycle_found = True
      break
    if state[next_node] == UNVISITED:
      dfs(next_node)
    # If already visited, ignore.
  # We have explored all neighbors
  state[cur_node] = VISITED

def has_cycle(n):
  state = [UNVISITED] * n
  for node in range(n):
    if state[node] == UNVISITED:
      dfs(node)
  return cycle_found
```

3 states

yellow

for each child

yellow → yellow → cycle!

grey

green.

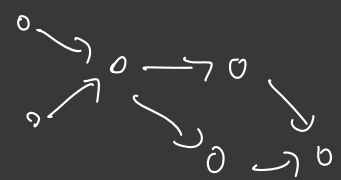
main entry

why call dfs on all?



① if graph is disconnected.

② if you start at the end

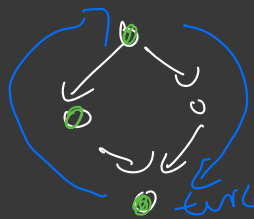


Find the bug

```
seen = set()
has_cycle = False

def dfs(u):
  if has_cycle:
    return
  seen.add(u)
  for v in adj[u]:
    if v in seen:
      has_cycle = True
      break
  dfs(v)

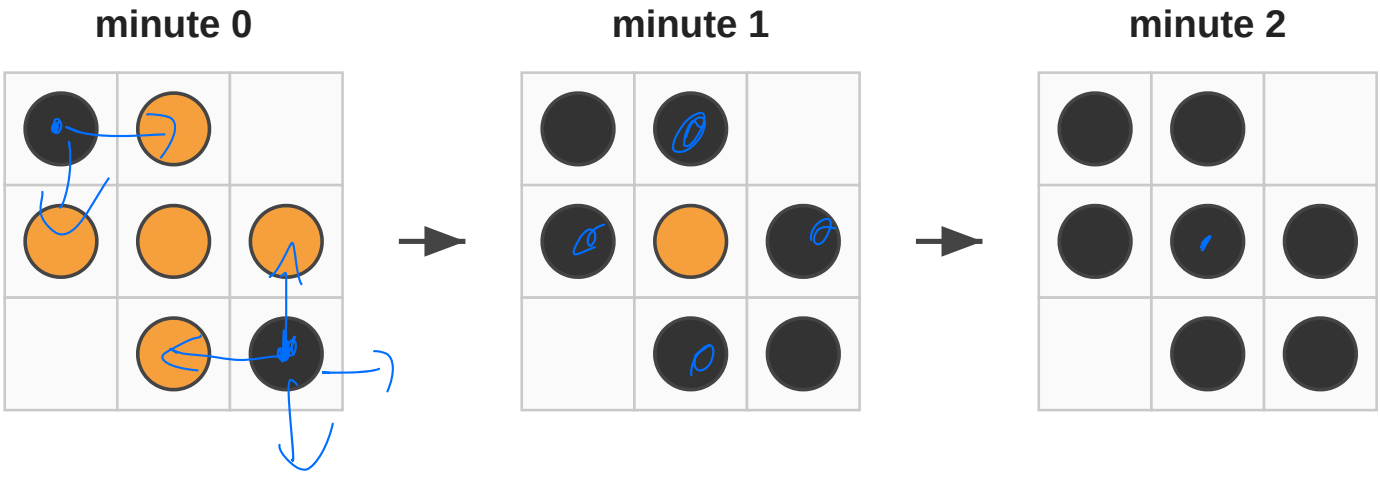
def has_cycle_buggy(n, adj):
  for v in range(n):
    if v not in seen:
      dfs(v)
  return has_cycle
```



cycle: saw a node twice

main entry

Rotting oranges



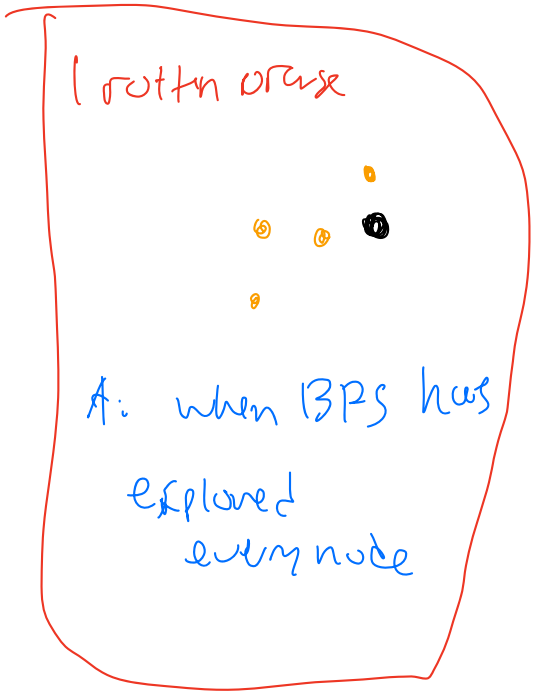
The problem

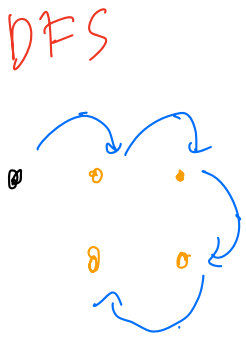
Input: an `R` x `C` grid. Each cell is empty (`0`), a **fresh** orange (`1`), or a **rotten** one (`2`).

Each minute, every fresh orange **4-adjacent** to a rotten one turns rotten.

Output: minutes until no fresh orange remains, or `-1` if that never happens.

Brainstorm







multiple rots? Q. multiple BFS

for each rotten orange
  for each timestamp
    set of infected oranges
ans: Union of all
= all oranges

02a. modified bfs

q: all rotten oranges

for each time step

infect neighbors

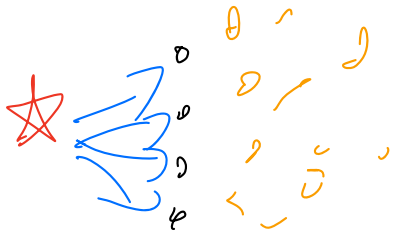
add it to q again

06.

node → time reached by each rotten orange



02b. BFS + supernode.



Runtime: V, E, R, C

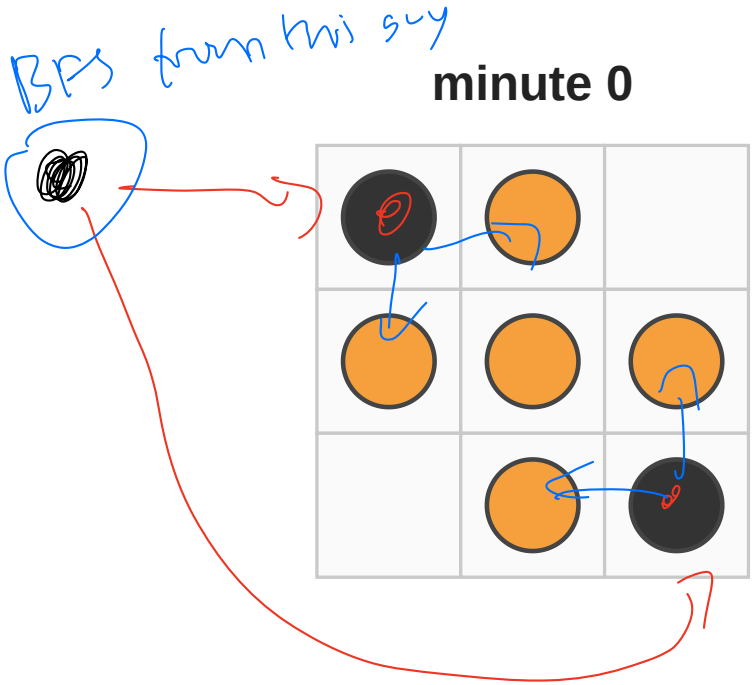
oranges, 4 directions

$O(V+E) \rightarrow O(R \times C)$

$O(R \times C)$ $O(4RC)$

$+ = O(RC)$

One imaginary source



Rotting oranges, in code

```python
from collections import deque
DIRS = ((0,1),(1,0),(0,-1),(-1,0))

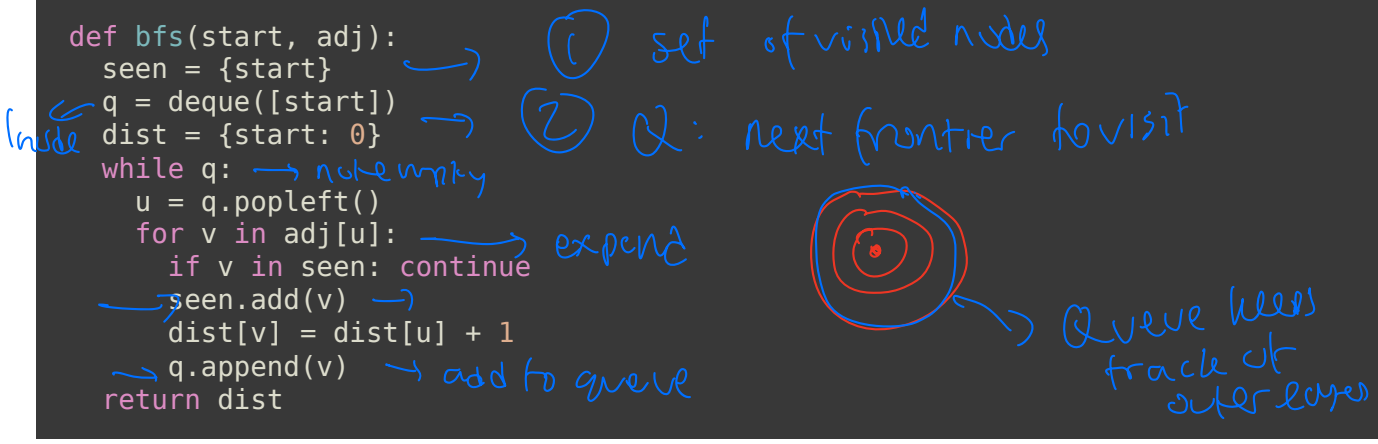
def oranges_rotting(grid):
  R, C = len(grid), len(grid[0])
  # seed with EVERY rotten orange at once
  q = deque((r,c) for r in range(R) for c in range(C) if grid[r][c] == 2)
  fresh = sum(row.count(1) for row in grid)

  minutes = 0
  while q and fresh:
    minutes += 1
    for _ in range(len(q)):
      r, c = q.popleft()
      for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if not (0 <= nr < R and 0 <= nc < C): continue
        if grid[nr][nc] != 1: continue
        grid[nr][nc] = 2
        fresh -= 1
        q.append((nr, nc))
  return -1 if fresh else minutes
```

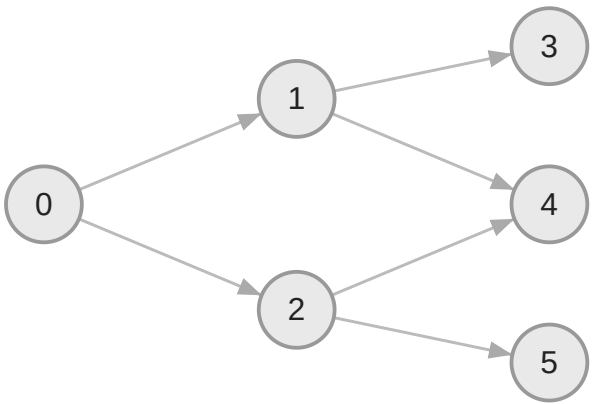
The BFS template

```
from collections import deque

def bfs(start, adj):
    seen = {start}
    q = deque([start])
    dist = {start: 0}
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v in seen: continue
            seen.add(v)
            dist[v] = dist[u] + 1
            q.append(v)
    return dist
```



BFS, one step at a time



not seen

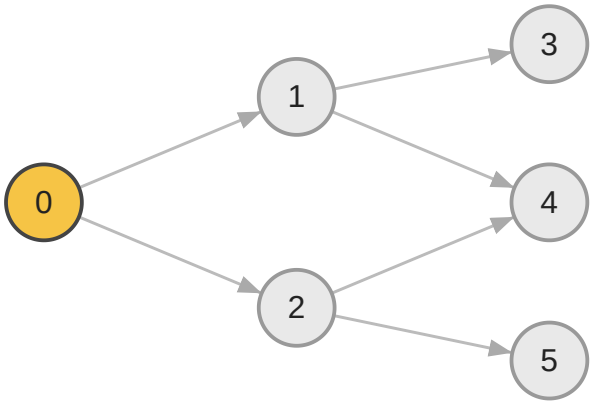
seen, waiting in queue

popped (processed)

queue (front → back)

(empty)

BFS, one step at a time



not seen

seen, waiting in queue

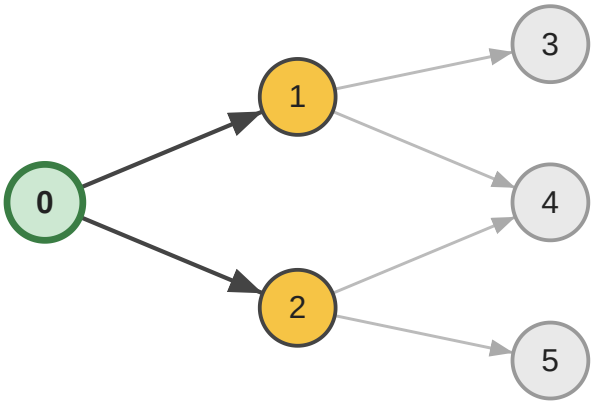
popped (processed)

queue (front → back)



d=0

BFS, one step at a time



not seen

seen, waiting in queue

popped (processed)

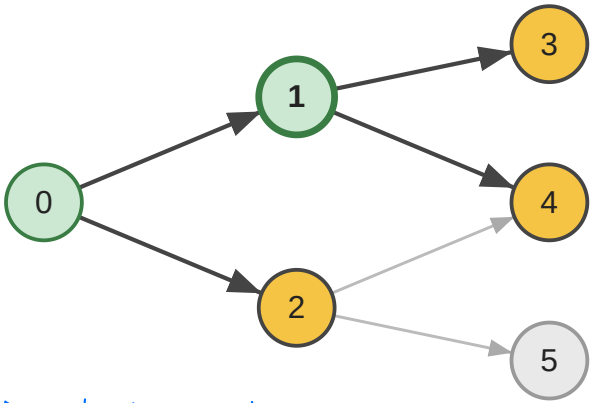
u = q.popleft() = **0**

queue (front → back)



d=1 d=1

BFS, one step at a time



not seen

seen, waiting in queue

popped (processed)

u = q.popleft() = **1**

queue (front → back)

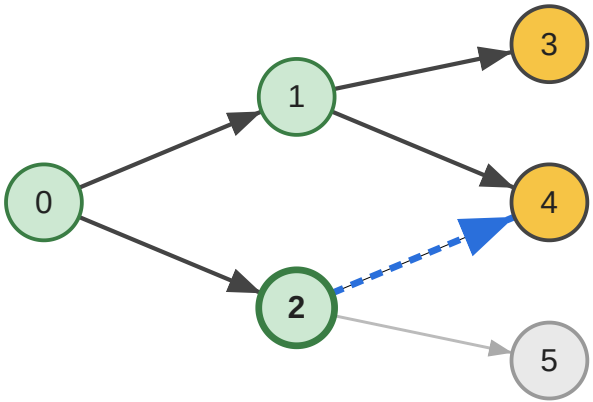
2	3	4
d=1	d=2	d=2

Q: how to know it's has been visited?

A: ① Set(NodeId) = Set(Integer), String

② node.state = 0/1/2... enum.

Already seen: skip it



not seen

seen, waiting in queue

popped (processed)

u = q.popleft() = **2**

queue (front → back)

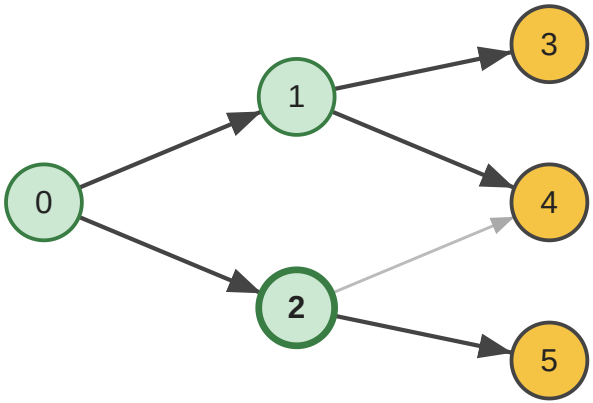


d=2 d=2

Q: If see same visited node, adjust distance?

A: never

BFS, one step at a time



not seen

seen, waiting in queue

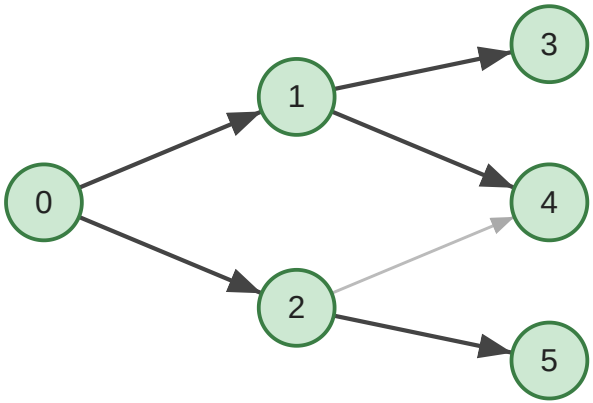
popped (processed)

u = q.popleft() = **2**

queue (front → back)



BFS, one step at a time



not seen

seen, waiting in queue

popped (processed)

queue (front → back)

(empty)

A BFS bug

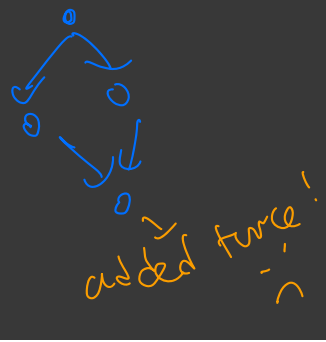
```
def bfs_buggy(start, adj):
  seen = set()
  q = deque([start])
  while q:
    u = q.popleft()
    if u in seen: continue
    seen.add(u)
    for v in adj[u]:
      if v not in seen:
        q.append(v)
```

start at 1 node

take node out

for each neighbor, add to q

seen.add(v)



Q: optimization?

A: put in seen before you add to the q

The 2D grid template

```
DIRS = ((0,1),(1,0),(0,-1),(-1,0))

for dr, dc in DIRS:
  nr, nc = r + dr, c + dc
  if not (0 <= nr < R and 0 <= nc < C): continue
  ...
```


A grid **is** a graph. `DIRS` is its adjacency list.

Default: DFS

BFS or DFS?

By default use **DFS**, except for **shortest path** or **level-ordered traversal**, then use **BFS**.

- **DFS**. cycles, connected components, tree-like traversal
- **BFS**. shortest path, level by level

Max area of island

1	1	0	0	1
1	1	1	0	0
0	0	0	1	1
1	0	0	1	0

The problem

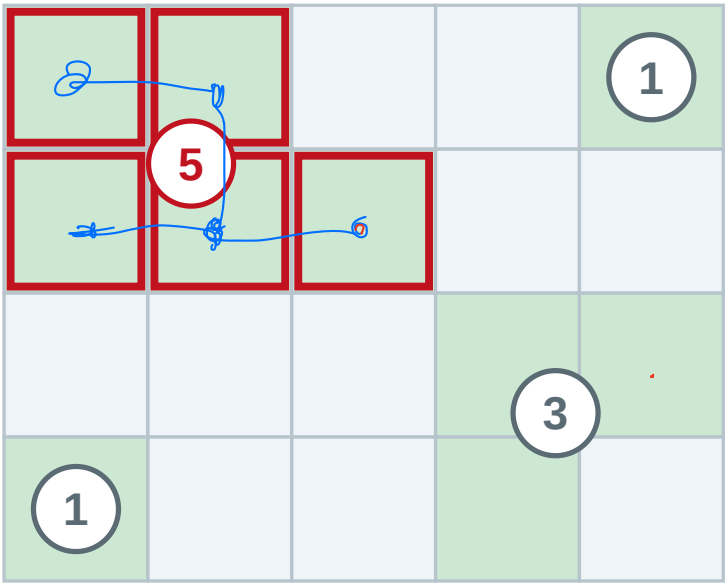
Input: an `R x C` grid of `0`s and `1`s.

An **island** is a group of `1`s connected 4-directionally. Its **area** is its number of cells.

Output: the area of the largest island, or `0` if there is none.

Each island is a connected component

Trees!



IDEAS

01 BFS / DFS Both work!

- start anywhere
-

Separate each island, then count?

IDEAS

- destroy island()
- connected components

↓ more generalized

for each cell
  if land
    recursively DFS(r,c)
    +
    islandcount++

Runtime ?

R, C.

At least O(R×C)

Ω(RXC)

↓

is it O(RC)? yes!

Point: conn component is O(V+E)

for any graph

Max area code

```
from collections import defaultdict
DIRS = ((0,1),(1,0),(0,-1),(-1,0))

grid = []
seen = set()
sizes = defaultdict(int)
R, C = 0, 0

def explore_island(r, c, comp_id):
  if not (0 <= r < R and 0 <= c < C): return
  if grid[r][c] == 0 or (r, c) in seen: return
  seen.add((r, c))
  sizes[comp_id] += 1
  for dr, dc in DIRS:
    explore_island(r + dr, c + dc, comp_id)

def max_area_of_island():
  R, C = len(grid), len(grid[0])
  comp_id = 0
  for r in range(R):
    for c in range(C):
      if grid[r][c] == 1 and (r, c) not in seen:
        explore_island(r, c, comp_id)
        comp_id += 1
  return max(sizes.values(), default=0)
```

cell traversed set?

island id → island size

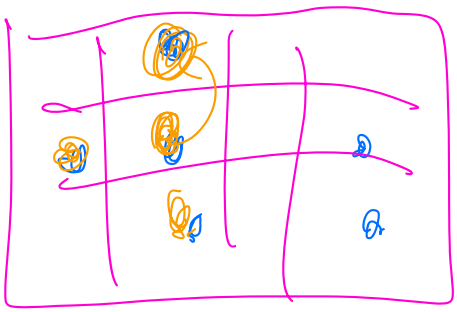
dfs → paint this island cells w/ comp_id

dfs

main entry point

for each cell, if not seen, & land explore it fully

island id



- componentId = 0
- map(islandId) → islandSize
- seen → set(boolean[])

compId = 1

map(islandId) → id=0 → size=4

seen = 4 coords

Why is one seen set enough here?

```
def explore_island(r, c, comp_id):
  if out of bounds or water: return
  if (r, c) in seen: return
  seen.add((r, c))
  for each neighbor:
    explore_island(...)
```

Grid traversal -- two states

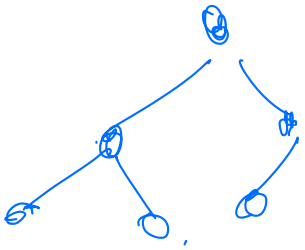
```
def dfs(u):
  state[u] = VISITING
  for v in adj[u]:
    if state[v] == VISITING: cycle!
    if state[v] == UNVISITED: dfs(v)
  state[u] = VISITED
```

Cycle detection -- three states

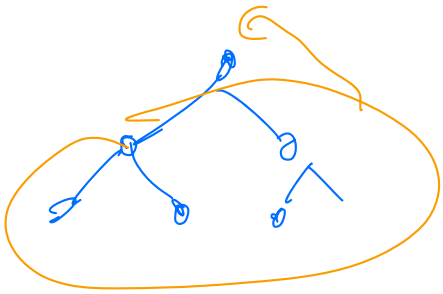
For DPS, when to use 2 vs 3?

DFS w/ 2 states

→ creating a tree.



BFS → creates a shortest path tree



Connected components code template

```
from collections import defaultdict

nodes = []
adj = defaultdict(list)
seen = set()
sizes = defaultdict(int)

def explore_component(node, comp_id):
  if node in seen: return
  seen.add(node)
  sizes[comp_id] += 1
  for nxt in adj[node]:
    explore_component(nxt, comp_id)

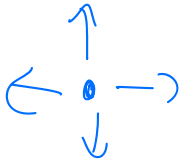

def connected_components():
  comp_id = 0
  for node in nodes:
    if node not in seen:
      explore_component(node, comp_id)
      comp_id += 1
  return comp_id
```

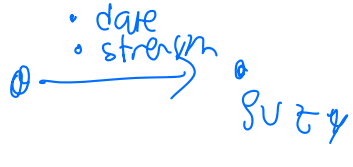
Tweak

Q: are these 2 cells on the same island?

A: check if eq island ids

Graph modeling practice





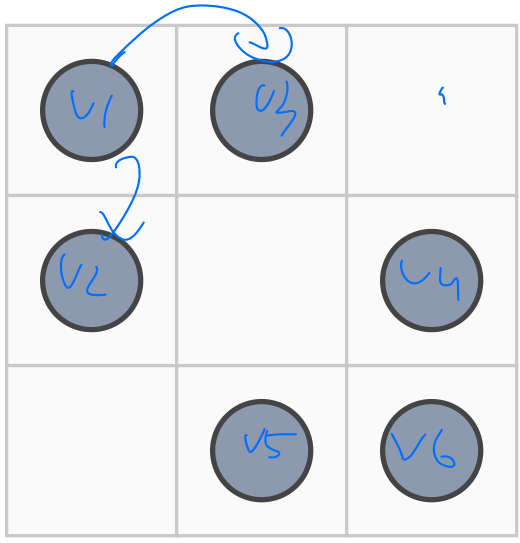
Two ways to hold a graph

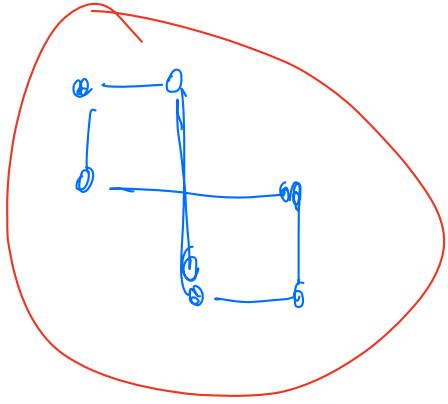
```python
# almost always this
from collections import defaultdict
adj = defaultdict(list)
adj[u].append(v)

# when the nodes and edges carry data
class Node:
  name: str
  edges: list["Edge"]

class Edge:
  to: Node
  weight: float
```

Most stones removed





Leetcode

Mapping to a graph

node : store

edges : $v_1 \rightarrow v_2$

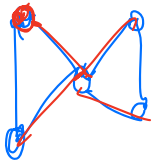
v_1

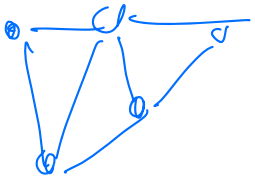
$\downarrow$

v_3

first node to my right / down.

1 connected component





Q: how many nodes are removable?

A: n − 1

Q: how? what's a traversal order that guarantees this?

A: ideas!

- sort nodes by # edges connected, take least



hint: create a tree out of this graph → BFS, DFS

Then cut leaves! → post order traversal → rev BFS

Q: high score

each connect comp → comp size − 1

Multiple connected components

- N = number of stones
- C = number of connected components
- R = stones removed

$R = \sum_{\text{island}} (\text{island size} - 1) = N - C$

$\sum_{\text{island}} \text{island size} - \sum_{\text{island}} 1$

$\leq N$

Pseudocode

```
1. Build the graph
   node = one stone
   edge between two stones sharing a row or a column

2. C = number of connected components

3. R = N - C
```

Evaluate division

$$\frac{a}{b} = 2$$

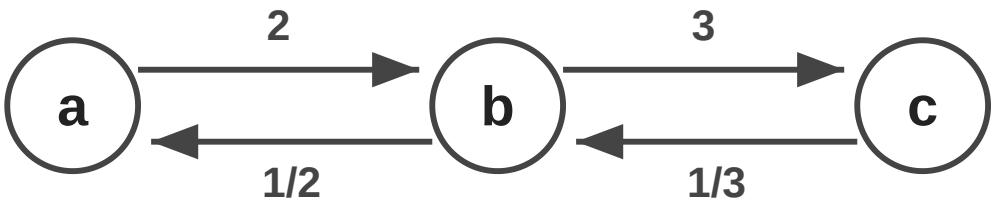
$$\frac{b}{c} = 3$$

$$\frac{a}{c} = ?$$

Leetcode

Evaluate division

$a/b = 2$ $\quad b/c = 3$



$c/b = \frac{1}{3}$

Algo: multiply edge weights of path $a \to c$.

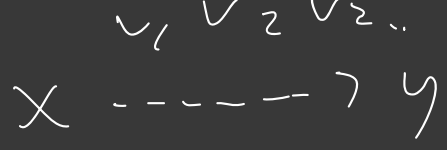
$2 \times 3 = 6$.

BFS, DFS ww

Pseudocode

```
1. Build the graph
   node = a variable name
   for each equation a / b = k:
     edge a -> b with weight k
     edge b -> a with weight 1 / k

2. For each query (x, y):
   if x or y was never seen: -1
   walk any path x -> y (BFS or DFS),
     multiplying the weights as you go
   no path: -1
```



Word ladder

```
Dict: [apple, bee, cog, dog, ...]
Query: "hit" -> "cog"?
```



Leetcode

Insight

Q1. Representation

Node – word in dict e.g. "hot"

edge – 2 words differ by 1 letter } both ways

Q2. Algo?

BFS from "a" "cog" ...

as soon as we see "hit" → done.

Building edges efficiently

01. Naive : each word → each word2, if differ by 1 → add edge.

W * W * L

02. smarter:

map (fingerprint) → list(word)

"h*t" → "hot", "hit"

W * (L * W)

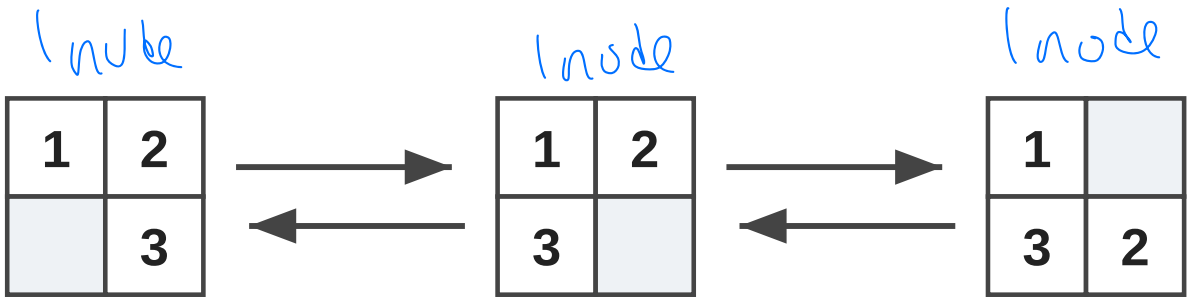
Pseudocode

```
1. Build the graph
   node = a word in the list, plus the start word
   edge between two words differing in exactly one letter

2. Answer = BFS distance from start to end, + 1
   we count words, not steps
   end word not reachable, or not in the list: 0
```

node: state
edges: states 1 move apart
also: BF

Brute-forcing game states



Leetcode

Pseudocode

```
1. node = one full board state
   edge = one legal move

2. Never build the graph.
   neighbors(state) generates them on demand

3. Answer = BFS distance from start state to goal state
   seen = set of states already reached
```

See 15-puzzle and Rush Hour

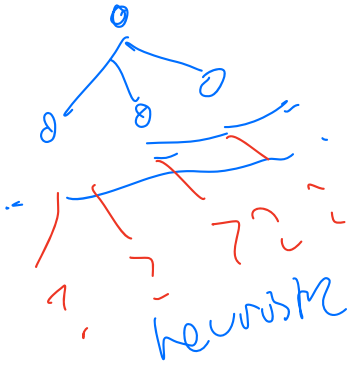
IRL of graph search.



01. BFS



02. A* search → heuristic.
dijkstra



03. Alpha, deepblue
↓
neural network.

many approach → heuristic

f(state) → score.

Recap

- BFS
- DFS
- Cycle detection
- Connected components